Week 7 — Delta Lake MERGE Implementation

Celebal Technologies — Data Engineer Internship

Palak Rathore

1. Objective

Perform incremental data processing using Delta Lake: load a customer master dataset into a Delta table, clean it, apply the Delta Lake MERGE operation against a simulated incremental batch of new and updated customers, and validate that the merge produced the correct result.

2. Dataset Description

Two related CSVs drive this assignment. customer_master.csv represents the existing customer table: 84 rows (80 unique customer_id values, 4 exact duplicates, 2 missing city values, and 3 missing email values — injected on purpose so the cleaning step has real work to do). customer_incremental.csv represents a new batch arriving for processing: 20 rows, of which 12 reference an existing customer_id with a changed city and/or status (an UPDATE), and 8 introduce brand-new customer_id values (an INSERT).

3. Implementation Approach

This implementation uses the official deltalake Python library (delta-rs, maintained by the Delta Lake project under the Linux Foundation). It reads and writes the real Delta Lake format — an actual _delta_log/ transaction log, Parquet data files, ACID commits, MERGE, and time travel — without requiring a Spark cluster, JVM, or Databricks workspace. Every result in this report is the actual output of a real run, not a simulated example.

A PySpark + delta-spark equivalent (the classic Databricks-style DeltaTable.forPath(spark, ...) API) is included as a reference appendix in the notebook, since that is the API most commonly taught for Delta Lake on Databricks. It was not executed in this environment because resolving the delta-spark jar requires Maven Central access, which this build sandbox's network doesn't reach — it will run unchanged on any machine or Databricks workspace with normal internet access.

4. Steps and Results

Step 1 — Load source dataset into a Delta table

```
write_deltalake(TABLE_PATH, df_master_raw, mode="overwrite")
dt = DeltaTable(TABLE_PATH)
```

Result: 84 rows loaded, Delta transaction log version 0 created.

Step 2 — Basic cleaning (nulls + duplicates)

Duplicates were dropped by customer_id (keep first), and nulls were filled: missing email → 'unknown@example.com', missing city → 'Unknown'. The cleaned data was committed as a new Delta version rather than mutating the original.

Check	Before	After
Row count	84	80
Duplicate customer_id rows	4	0
Missing city	2	0
Missing email	3	0

Delta version after this commit: 1.

Step 3 — Load the incremental dataset

customer_incremental.csv (20 rows) was read and diffed against the existing customer_id set: 12 rows matched an existing customer (destined for UPDATE), 8 did not (destined for INSERT).

Step 4 — Apply the MERGE operation (SCD Type 1)

```
dt.merge(
    source=df_incremental,
    predicate="t.customer_id = s.customer_id",
    source_alias="s", target_alias="t",
).when_matched_update_all().when_not_matched_insert_all().execute()
```

Real metrics reported by the Delta transaction log for this single atomic commit (dt.history()):

Metric	Value
num_target_rows_updated	12
num_target_rows_inserted	8
num_target_rows_copied (unchanged rows)	68
num_output_rows	88

Delta version after the MERGE: 2. This is one of Delta Lake's core guarantees — the entire MERGE (12 updates + 8 inserts across the whole table) either commits as a single atomic version, or not at all; there is no partially-applied intermediate state a concurrent reader could see.

Step 5 — Validation

Check	Result	Status
Total row count = 80 existing + 8 new	88 == 88	PASS
Duplicate customer_id in final table	0	PASS
CUST0035 reflects new city/status (Chennai/Trial)	Chennai/Trial	PASS
New customer CUST0081 present in final table	present	PASS

Step 6 — Final table + summary

The final Delta table holds 88 rows with 0 duplicate customer_id values. dt.history() shows all three commits made during this run — version 0 (initial load, 84 rows), version 1 (cleaning, 80 rows), and version 2 (MERGE, 88 rows) — each independently queryable via Delta's time-travel feature (spark.read.format('delta').option('versionAsOf', N).load(...) or the equivalent deltalake API).

5. Bonus — SCD Type 2 (History-Preserving MERGE)

The suggested screenshot folder structure for this assignment included both scd1/ and scd2/ subfolders, implying a second, history-preserving flavor of the merge was also expected. This was implemented on a separate Delta table (customers_scd2) so it doesn't interfere with the core SCD1 deliverable above.

- Phase A (expire): MERGE sets is_current = False and end_date = today on the OLD row of every customer being updated (12 rows expired).
- Phase B (insert): append a brand-new row for every incoming record — update or new — with is_current = True, effective_date = today.

Metric	Value
Total historical rows (all versions)	100
Currently active rows (is_current = True)	88
Duplicate customer_id among current rows	0

Example — CUST0035 now has two rows in the SCD2 table:

city	status	effective_date	end_date	is_current
Mumbai	Active	2024-09-20	2026-08-03	False
Chennai	Trial	2026-08-03	(none)	True

The SCD Type 1 table only ever shows the second row — the fact that this customer used to be in Mumbai with Active status is gone the moment the MERGE commits. The SCD Type 2 table keeps both, at the cost of the table growing with every subsequent update instead of staying a fixed size.

6. Conclusion

This assignment demonstrates the core value Delta Lake adds over plain Parquet or CSV for incremental processing: an atomic MERGE that combines an UPDATE and an INSERT into a single all-or-nothing commit, a queryable transaction log for auditing exactly what changed and when, and — as the SCD2 bonus shows — a foundation flexible enough to support full history-preserving change tracking without a fundamentally different toolset. Every number in this report came from an actual executed run against real generated data, not a hypothetical walkthrough.